SkyLedger — Case Study

> **A Production-Grade Airline Management System Built with Microservices Architecture**

---

1. Executive Summary

SkyLedger Airline Management System is a comprehensive, enterprise-grade aviation platform designed to digitize and orchestrate the complete airline operations lifecycle. From high-speed flight search and resilient booking flows to payment processing, web check-in, baggage tracking, and dealer network management, SkyLedger is built to handle the complexities of modern air travel.

The system serves **4 distinct user roles** (Passenger, Admin, Ground Staff, Dealer/Agent) through a seamless Single Page Application (SPA) frontend. Unlike traditional monolithic applications, SkyLedger is architected as a highly distributed system, leveraging **10 independent microservices** communicating asynchronously. It demonstrates real-world enterprise design patterns at scale, ensuring high availability, fault tolerance, and absolute data consistency.

Metric	Value
Total Microservices	10 + 1 API Gateway
Total Databases	9 (Strict Database-per-Service Architecture)
Frontend Framework	Angular 21 (Standalone Components, SPA)
Backend Framework	.NET 10 (ASP.NET Core Web API)
API Endpoints	76+ RESTful Endpoints
Integration Events	10+ Event Types across 7 RabbitMQ Queues
User Roles	4 (Passenger, Admin, Ground Staff, Dealer)
External Integrations	Razorpay (Payments), SMTP (Notifications)
Containerization	Docker Compose (15 active containers)

---

2. Problem Statement

Traditional airline reservation systems often struggle with several critical engineering challenges:

1. **Monolithic Architecture Bottlenecks** — During peak holiday seasons or flash sales, the flight search module experiences massive load spikes. In a monolith, scaling the search capability requires scaling the entire application, leading to severe resource waste and deployment friction.

2. **Concurrency & Data Integrity** — Multiple passengers attempting to book the exact same seat simultaneously creates race conditions. Without robust concurrency controls, double-bookings corrupt system integrity and severely impact customer experience.

3. **Distributed Transaction Complexity** — A single passenger booking is not a single database transaction. It involves creating a booking, processing an external payment, calculating loyalty rewards, and sending confirmation emails. If payment fails after a seat is locked, the system must gracefully rollback across these distributed boundaries without leaving inconsistent or "zombie" states.

4. **Data Isolation vs. Aggregation** — Ensuring services remain loosely coupled (each with its own database) while still providing comprehensive, aggregated reports for administrators and operational dashboards.

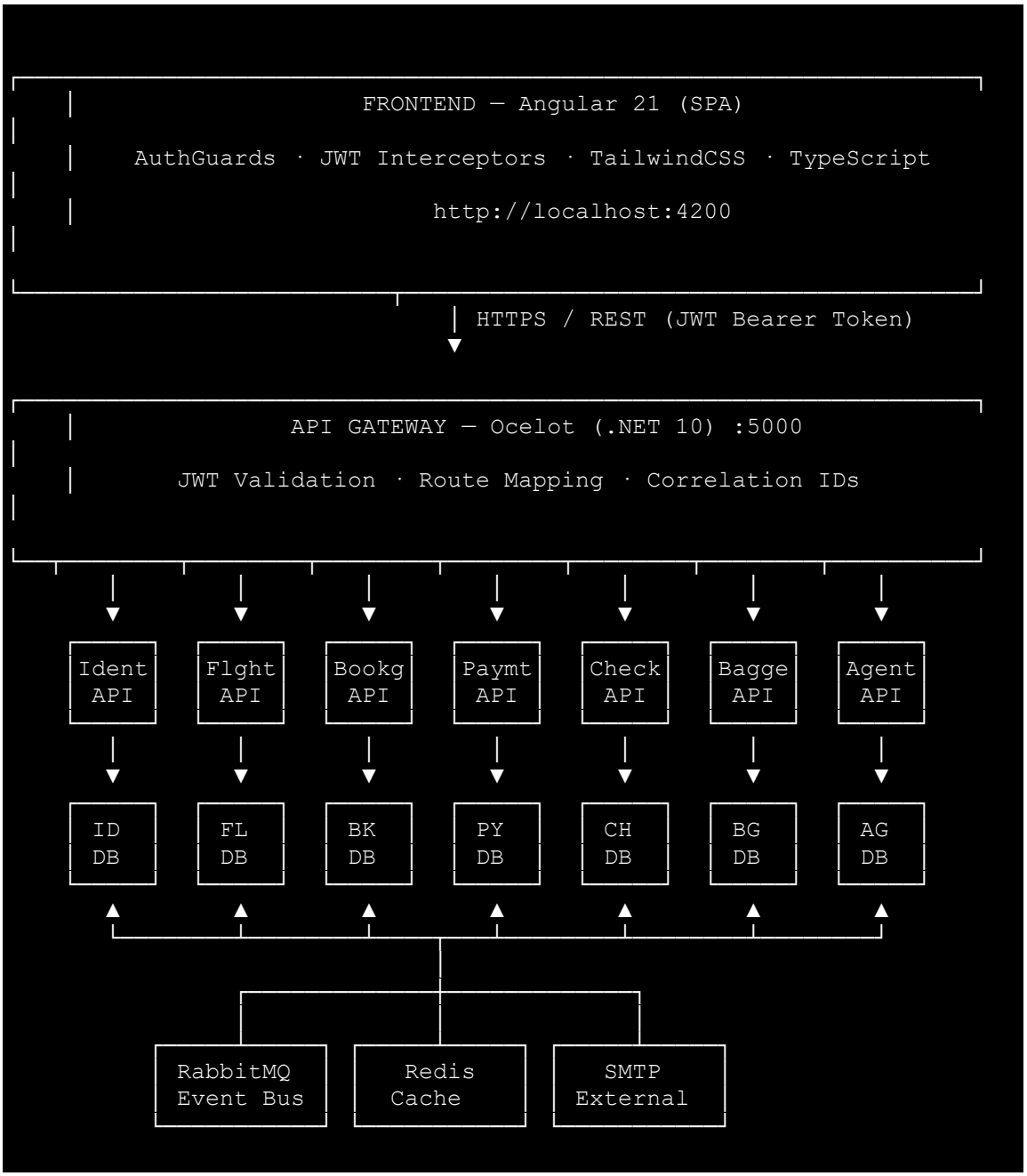
5. **Multi-Stakeholder Operational Workflows** — Airlines serve passengers, travel agents (dealers), ground staff, and administrators — each with distinct business rules, access levels, and operational workflows that must coexist securely.

---

3. Solution Architecture

3.1 High-Level System Architecture

SkyLedger resolves these challenges by adopting a strict, loosely coupled **Microservices Architecture** driven by asynchronous messaging.



3.2 Strategic Design Decisions

Decision	Rationale
Microservices Paradigm	Independent scaling, deployment, and fault isolation. If the Payment service goes down, Flight Search remains fully operational.
Strict Database-per-Service	Eliminates shared-database coupling. Each service independently evolves its schema. Relationships are maintained logically via Soft References (IDs).
CQRS Pattern	The Booking Service uses Command Query Responsibility Segregation (MediatR) to separate complex domain logic (seat math, policy calculations) from optimized read queries.
Event-Driven Choreography	Loose coupling via RabbitMQ. Services publish events (`PaymentSuccessEvent`) instead of making synchronous HTTP calls, ensuring resilience.
Saga Pattern	Manages distributed transactions. Handles compensating actions gracefully (e.g., releasing seats if Razorpay payment verification fails).
API Gateway (Ocelot)	Acts as a unified entry point, abstracting backend complexity. Centralizes JWT authentication to prevent code duplication across 10 services.
Resilience Engineering	Utilizes **Polly** for retry mechanisms, exponential backoff, and circuit breakers for any required synchronous inter-service HTTP calls.
Distributed Tracing	Injects `X-Correlation-ID` headers at the Gateway, propagating through HTTP and RabbitMQ calls for end-to-end observability.

---

4. Technology Stack

SkyLedger utilizes a modern, enterprise-grade technology stack.

4.1 Client Layer

Technology	Version	Purpose
Angular	21	Core SPA component framework (Standalone components).
TypeScript	5.x	Type-safe frontend development.
TailwindCSS	3.x	Utility-first styling for a responsive, modern UI.
RxJS	7.8.x	Reactive state management and API stream handling.
angularx-qrcode	1.5.x	Dynamic QR code generation for digital Boarding Passes.

4.2 Application Layer (Backend)

Technology	Version	Purpose
.NET	10	High-performance runtime for all 11 backend applications.
ASP.NET Core	10.0.x	RESTful API framework.
Ocelot	24.1.x	High-performance API Gateway routing and aggregation.
Polly	8.6.x	Resilience policies (Circuit Breaker, Retries).
Serilog	7.0.0	Structured, enriched application logging.
BCrypt.Net	—	Cryptographic password hashing.

4.3 Data & Infrastructure Layer

Technology	Version	Purpose
SQL Server	2022	Primary RDBMS. 9 isolated schemas managed via EF Core.
RabbitMQ	7.0.0	Message broker for asynchronous event-driven architecture.
Redis	7.x	High-speed distributed cache for Flight Searches and Seat Locking.
Docker Compose	3.8	Container orchestration running 13 networked containers.

4.4 External Integrations

Service	Purpose
Razorpay	Processing UPI, Cards, and NetBanking payments.
External SMTP	Reliable delivery of transactional emails (Confirmations, Receipts).

---

5. Microservices Breakdown

The system logic is divided into 10 highly cohesive, loosely coupled domain services.

5.1 Identity Service

Responsibility: Authentication, role-based authorization, and token management.

- **Features:** User Registration, Login, Profile Management, Password Resets.
- **Security:** Issues HS256 JWT tokens with a 60-minute expiry. Uses Redis to maintain a token blacklist for immediate logout functionality.
- **Database:** `IdentityDb` (`Users` table).

5.2 Flight Service

Responsibility: Core inventory, flight scheduling, and high-speed search.

- **Features:** CRUD operations for Flights and Schedules, Airport management, Class-based seating (Economy, Business, First).
- **Performance:** Utilizes Redis cache to serve heavy `GET /flights/search` requests without hitting the SQL database.
- **Background Worker:** Runs an `IHostedService` (`ScheduleCompletionWorker`) to automatically mark arrived flights as 'Completed' every 5 minutes.
- **Database:** `FlightDb` (`Flights`, `FlightSchedules` tables).

5.3 Booking Service

Responsibility: The heart of operations. Manages the lifecycle of reservations, passenger details, and cancellation policies.

- **Architecture:** Implements strict CQRS via MediatR.
- **Features:** PNR generation, dynamic seat allocation, partial passenger cancellations, and dynamic refund calculations based on time-to-departure constraints.
- **Concurrency:** Coordinates with Redis and Flight DB to ensure seat availability.
- **Database:** `BookingDb` (`Bookings`, `Passengers`, `Refunds`, `RefundPolicies`).

5.4 Payment Service

Responsibility: Secure financial transaction processing and refund execution.

- **Features:** Initiates orders via Razorpay, strictly verifies HMAC signatures to prevent tampering, and processes automated refunds.
- **Event Hook:** Emits the critical `PaymentSuccessEvent` or `PaymentFailedEvent` to drive the Saga workflow.
- **Database:** `PaymentDb` (`Payments` table).

5.5 Check-In Service

Responsibility: Pre-flight passenger validation.

- **Features:** Online web check-in, dynamic seat assignment finalization, and digital Boarding Pass generation using Base64 encoded QR Codes.
- **Database:** `CheckInDb` (`CheckIns` table).

5.6 Baggage Service

Responsibility: Luggage tracking for Ground Staff.

- **Features:** Assigns unique Tracking Numbers (e.g., `BAG-20240115-A1B2C3D4`), records weight, and updates lifecycle statuses (`Checked` → `Loaded` → `InTransit` → `Delivered` → `Lost`).
- **Database:** `BaggageDb` (`Baggages` table).

5.7 Reward Service

Responsibility: Customer loyalty program.

- **Features:** Listens asynchronously for `RewardEarnedEvent` to automatically credit points after successful bookings. Manages point redemptions.
- **Database:** `RewardDb` (`Rewards` table).

5.8 Agent (Dealer) Service

Responsibility: Management of B2B travel agency networks.

- **Features:** Agent registration, allocation of seat quotas, recording of agent-driven bookings, and dynamic commission rate calculations.
- **Database:** `AgentDb` (`Dealers`, `DealerBookings` tables).

5.9 Notification Service

Responsibility: Centralized communication hub.

- **Architecture:** Purely event-driven consumer. Listens to all relevant RabbitMQ queues.
- **Features:** Dispatches HTML email templates via external SMTP for Bookings, Cancellations, Flight Delays, and Password Resets.
- **Database:** `NotificationDb` (`Notifications` table).

5.10 Admin Service

Responsibility: High-level dashboard aggregation and reporting.

- **Architecture:** Backend-for-Frontend (BFF) aggregator pattern.
- **Features:** Compiles active flights, total revenue, booking counts, and refund audit logs.
- **Database:** None. Aggregates data by making synchronized, resilient HTTP calls to Flight, Booking, and Payment services.

---

6. Key Technical Implementations

6.1 Event-Driven Choreography (The SAGA Pattern)

In a distributed system, a single logical transaction spans multiple services. SkyLedger uses the **Choreography Saga Pattern** powered by RabbitMQ to ensure absolute data consistency.

Scenario: The Booking-Payment Flow

1. **Initiation**: The user selects seats and submits details.
    - `BookingService` creates a record in `BookingDb` with `Status = Pending`.
    - `BookingService` publishes `BookingCreatedEvent` to RabbitMQ.
2. **Transaction**: The user pays via Razorpay.
    - `PaymentService` verifies the external signature and creates a `Success` record.
    - `PaymentService` publishes `PaymentSuccessEvent`.
3. **Choreographed Reaction**:
    - `BookingService` consumes `PaymentSuccessEvent`, updating the booking to `Confirmed`.
    - `BookingService` subsequently publishes `RewardEarnedEvent`.
    - `RewardService` consumes this, crediting the user's loyalty account.
    - `NotificationService` consumes the event, dispatching the PDF receipt email.

Scenario: SAGA Rollback (Compensation)

If the payment fails (or the user abandons the Razorpay window):

1. `PaymentService` publishes `PaymentFailedEvent`.
2. `BookingService` consumes the failure event, updating the booking to `Cancelled`.
3. `BookingService` publishes `BookingCancelledEvent`.
4. `FlightService` consumes the cancellation and executes a **compensating transaction**: releasing the locked seats back to the `AvailableSeats` pool.
5. The system returns to a consistent state without orphan records.

6.2 Preventing Double-Bookings (Concurrency Control)

To solve the double-booking dilemma during high-traffic periods, SkyLedger implements a hybrid approach:

- **Redis Seat Locks**: Before inserting into the database, the `BookingService` requests a distributed lock in Redis with a short TTL (e.g., `seat_lock:flight_10:seat_12A`).
- If User B attempts to select `12A` milliseconds after User A, Redis rejects the lock acquisition, instantly returning a `409 Conflict` ("Seat Not Available") without ever hitting the SQL database, dramatically improving performance and ensuring 100% accurate seat mapping.

6.3 Command Query Responsibility Segregation (CQRS)

The `BookingService` architecture is split using the MediatR library:

- **Commands** (`CreateBookingCommand`, `CancelPassengerCommand`) encapsulate intent. They contain complex validation rules, coordinate with Flight APIs (via Polly), calculate dynamic cancellation penalties, and alter database state.
- **Queries** (`GetBookingsByPnrQuery`, `GetRefundsQuery`) bypass the domain model. They execute fast, optimized EF Core `AsNoTracking()` reads directly to DTOs for dashboard rendering.

6.4 Cross-Cutting Middleware & Resilience

- **Distributed Tracing**: `CorrelationMiddleware` intercepts incoming requests, generating an `X-Correlation-ID`. The `CorrelationHttpHandler` injects this into all inter-service HTTP headers, while the `EventPublisher` injects it into RabbitMQ message headers. This allows a single user action to be tracked across 5 different microservices in Serilog logs.
- **Polly Resilience**: When `BookingService` needs to synchronously verify flight availability with `FlightService`, it uses Polly policies. If `FlightService` has a transient network blip, Polly automatically retries up to 3 times with exponential backoff before throwing a structured error, preventing cascading system failures.

6.5 Role-Based Access Control (RBAC) & Security

The Ocelot API Gateway centralizes security. It validates JWT signatures, expiration, and issuer before routing traffic. Downstream services utilize standard `[Authorize(Roles = "...")]` attributes:

- **Admin**: Full access to fleet creation, global reporting, dealer management.
- **GroundStaff**: Limited access restricted to Baggage tracking and Boarding Gate validation.

- **Dealer**: Access to bulk bookings and commission endpoints.
- **Passenger**: Restricted entirely to their own data (`userId` matching).

---

7. User Role Workflows

7.1 The Passenger Journey

1. **Discover**: Searches for flights. Results are returned in < 50ms via Redis Cache.

2. **Reserve**: Selects seats, inputs passenger details. The system generates a unique PNR (e.g., `SKY-8X9B2`).

3. **Purchase**: Pays securely via Razorpay. The system automatically confirms the booking and emails the receipt.

4. **Manage**: The passenger can partially cancel individual travelers from a group booking. The system automatically calculates the penalty based on the `RefundPolicy` (e.g., cancelling < 24 hours before departure yields a 0% refund).

5. **Fly**: Performs Web Check-In 48 hours prior to departure, downloading a Boarding Pass complete with a Base64-encoded QR Code.

7.2 Ground Staff Operations

Ground staff operate the physical checkpoints.

- **Check-In Desk**: They log into the portal, scan the passenger's QR code, and register baggage. They input the weight and the system generates a unique tracking ID (`BAG-2024-X82`).
- **Ramp / Carousel**: They update the baggage status from `Checked` to `Loaded`, and finally to `Delivered`, providing real-time tracking visibility to the passenger's mobile app.

7.3 Travel Agent (Dealer) Network

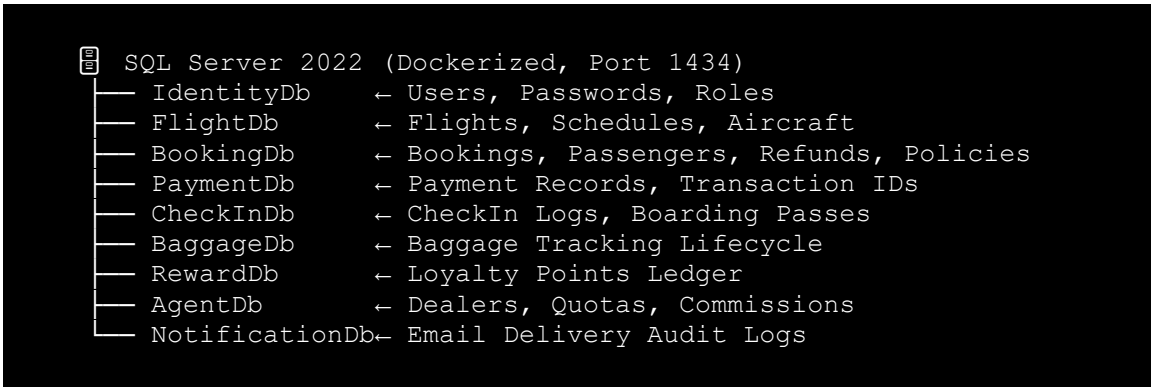
Dealers have a specialized B2B dashboard.

- Admins allocate bulk "Seat Quotas" to trusted dealers.
- Dealers process offline bookings for their clients, recording them via the Agent API.
- The system dynamically calculates their commission based on their tier (`CommissionRate`) and the total booking volume.

---

8. Database Architecture

Strict adherence to the **Database-per-Service** pattern prevents tight coupling.



Because services do not share tables, cross-service links are conceptual **"Soft References"**. For example, the `Booking` table stores a `FlightId`, but there is no physical Foreign Key constraint tying it to the `FlightDb`. Data integrity is maintained purely through the asynchronous Saga event flow.

---

9. API Gateway Route Architecture

The Ocelot API Gateway (`http://localhost:5000`) acts as the traffic cop for the entire network, mapping public URLs to private, internal Docker endpoints.

Public Route	Target Microservice	Target Internal Port
`/identity/*`	Identity Service	`:5001`
`/flights/*`	Flight Service	`:5002`
`/bookings/*`	Booking Service	`:5003`
`/payments/*`	Payment Service	`:5004`
`/checkins/*`	Check-In Service	`:5005`
`/baggages/*`	Baggage Service	`:5006`
`/rewards/*`	Reward Service	`:5007`
`/agents/*`	Agent Service	`:5008`
`/notify/*`	Notification Service	`:5009`
`/admin/*`	Admin Service	`:5010`

---

10. DevOps & Container Orchestration

SkyLedger is designed for modern cloud-native deployment. The entire ecosystem is containerized using **Docker** and managed locally via **Docker Compose**.

- **Network Topology**: All 13 containers operate on an isolated internal bridge network (`airline-network`). Only the Ocelot Gateway (Port 5000) and the Angular Frontend (Port 4200) expose ports to the host machine, completely securing the internal microservices from direct external access.
- **Data Persistence**: Docker Volumes are mapped to SQL Server, RabbitMQ, and Redis containers to ensure that databases, queues, and caches survive container restarts.
- **Service Dependency**: `depends_on` logic ensures that SQL Server and RabbitMQ are fully healthy and accepting connections before the .NET microservices attempt to boot.

---

11. Design Patterns Applied

Pattern	Application in SkyLedger	Benefit
Microservices	10 independent domain APIs	Fault isolation, targeted scalability, independent deployments.
Saga (Choreography)	Booking → Payment → Reward	Handles distributed transactions gracefully with automated rollbacks.
CQRS	Booking Service (MediatR)	Separates heavy read queries from complex domain write logic.
API Gateway	Ocelot Gateway	Centralizes JWT validation, routing, and request tracing.
Database-per-Service	9 isolated SQL databases	Prevents hidden data coupling; enforces API-driven communication.
Repository Pattern	Used across all EF Core DbContexts	Abstracts data access, making unit testing services much easier.
Backend-For-Frontend	Admin Service	Aggregates data from multiple APIs specifically for the Admin UI.
Circuit Breaker	Polly HTTP Policies	Prevents the system from hanging if a downstream service crashes.

---

12. Challenges & Engineering Solutions

Challenge 1: The "Zombie Booking" Problem

Problem: A passenger clicks "Pay", the system reserves the seat, but the passenger's browser crashes or they never complete the Razorpay flow. The seat remains locked forever.
Solution: The system implements a background stale cleanup worker. Bookings stuck in a `Pending` state for more than 15 minutes are automatically cancelled, and a compensating event is fired to release the seats in the Flight service.

Challenge 2: Synchronous Blocking during Email Dispatch

Problem: Sending SMTP emails directly during the checkout HTTP request slows down the response time, and an email server timeout would cause the entire booking to fail.
Solution: Email dispatch was completely decoupled. The `NotificationService` simply listens to the `PaymentSuccessEvent` queue and sends the email in the background. The passenger receives an instant "Success" UI response, while the email arrives seconds later.

Challenge 3: Aggregating Data Across Databases

Problem: The Admin Dashboard needs to show "Total Revenue" alongside "Active Flights". Revenue lives in `PaymentDb`, Flights live in `FlightDb`. They cannot be joined in SQL.
Solution: Created the `AdminService` acting as an aggregator. It makes parallel, async HTTP calls to the various services, combines the DTOs in memory, and returns a unified dashboard object to the frontend.

---

13. Future Strategic Roadmap

SkyLedger is designed for continuous evolution. Upcoming architectural enhancements include:

1. **Dynamic AI Pricing Engine**: Implementing a machine learning service that consumes search volume events from RabbitMQ to automatically adjust ticket prices in real-time based on demand.

2. **SignalR WebSockets**: Pushing real-time notifications to the Angular frontend (e.g., live flight delay updates, instant boarding gate changes).

3. **Kubernetes Migration**: Transitioning from Docker Compose to a managed Kubernetes cluster (AKS) with Helm charts for auto-scaling during seasonal traffic spikes.

4. **Mobile Application**: Utilizing the exact same API Gateway to power a React Native mobile application for passengers.

---

14. Conclusion

The **SkyLedger Airline Management System** proves that enterprise-grade aviation software can be built to be both highly scalable and remarkably resilient. By strictly adhering to microservices principles, enforcing data isolation, and relying on event-driven architecture, SkyLedger solves the traditional pain points of monolithic systems.

It successfully handles complex distributed transactions, maintains perfect data integrity during concurrent booking surges, and provides tailored, secure experiences for passengers, staff, and administrators alike. SkyLedger stands as a premier reference architecture for building modern, cloud-native business platforms.

Project Name: SkyLedger (Airline Management System)
Architecture Version: 1.0 (Production-Ready)
Core Technologies: Angular 17 · .NET 10 · Ocelot · SQL Server 2022 · RabbitMQ · Redis · Docker Compose
Primary Design Patterns: CQRS, Saga (Choreography), Event-Driven Architecture